

Week 2 Day 1

A. Doniga

Theory:

1) Control flow is the order in which statements and instructions are executed in a program. It allows a program to make decisions and repeat or skip operations using structures such as if/else, switch, loops, break, and continue.

2) if/else if is suitable for checking different conditions, especially when conditions involve ranges, comparisons, or complex expressions. switch is preferable when comparing one expression against multiple fixed values, because it is often easier to read and maintain.

Example situations:

* if/else if $\rightarrow$ checking $\text{score} \geq 90$, $\text{score} \geq 80$, etc.

* Switch $\rightarrow$ checking whether day is "Monday", "Tuesday", etc.

3) break terminates the switch statement after the matching case is executed.

If break is omitted, JavaScript continues executing the following cases until it encounters a break or reaches the end of the switch. This is called fall-through.

4) A nested if is an if statement placed inside another if statement. It can become a code smell when there are many levels of nesting because the code becomes difficult to read, test, debug and maintain. Early returns, guard clauses or combining conditions can often reduce unnecessary nesting.

5) break completely terminates the loop.

Continue skips the current iteration and moves to the next iteration.

For example, if a loop processes numbers from 1 to 10:

break $\rightarrow$ stops the loop completely.

Continue $\rightarrow$ skips the current number but continues the loop.

6) Yes, A switch can intentionally fall through by omitting break between cases. This is useful when multiple cases should perform the same action.

Eg)

case "Saturday":

case "Sunday":

console.log("Weekend");

break;

Both cases execute the same block.

$\rightarrow$ The default case executes when none of the specified case value matches the switch expression. It is similar to the final else in an if/else structure.

8) Javascript switch uses strict equality ($===$) when ~~comp~~ comparing the switch expression with case values. Therefore both value and data type must match.

9) The common bug is unintended fall-through. After a matching case executes, Javascript continues executing the subsequent cases until a break is encountered or the switch ends. This can cause multiple actions to execute unexpectedly.

10) use else if when conditions involve,

• Ranges

- Comparisons such as $>$, $<$, $=$
 - Multiple variables
 - Complex Boolean expressions
 - Conditions that are not simply fixed-value matches
- For example, checking grades based on score ranges is more naturally handled with `else if`.

Output prediction:

- 11) big
- 12) two
three
- 13) A
- 14) 0
1
2
4
- 15) 0
1
2
- 16) no match
- 17) false
- 18) 0
1
2
- 19) loose equal
- 20) Fung Bungy

Error Finding:

- 21) The break is missing after case 1.

O/p: One
two

Because there is no break, execution falls through from

case 1 to case 2.

Correct Ans:

```
switch (1) {
```

```
  case 1:
```

```
    console.log("one");
```

```
    break;
```

```
  case 2:
```

```
    console.log("two");
```

```
    break;
```

```
}
```

22) The code uses the assignment operator = instead of a comparison operator. $x = 5$ assigns 5 to x . Since 5 is truthy, the if block executes. If the intention is to compare the value, use.

```
if (x === 5) {
```

```
  console.log("Correct");
```

```
}
```

23) There is no match because "1" is a string, whereas 1 is a number. Switch uses strict equality (===).

"1" === 1 // false.

24) This causes a Reference Error because i was declared using let inside the for loop. let is block-scoped, so it cannot be accessed outside the loop.

25) If a and b have not been declared, JavaScript throws a Reference Error.

For eg)

Reference Error: a is not defined

Both variables must be declared ~~for~~ before they are used.

2b) Switch (true) is a valid but less common pattern. Each case Expression is evaluated, and its Boolean result is compared with true.

For Eg):

```
Switch (true) {
```

```
  case x > 10:
```

```
    console.log("Greater than 10");
```

```
    break;
```

```
  case x > 5:
```

```
    console.log("Greater than 5");
```

```
    break;
```

```
}
```

It can be used to represent condition-based logic, but normal if / else if is often more readable.

2c) When nested if statements do not use braces, an else is associated with the nearest unmatched if.

Eg). if (a)

if (b)

console.log("B");

else

console.log("B");

The else belongs to:

if (b)

not:

if (a)

Using braces prevents this ambiguity

29) Using break outside loop or switch causes a Syntax Error. break must be associated with an enclosing loop or switch statement.

30) Using continue outside a loop causes a Syntax Error because continue is only valid inside loop statements such as for, while or do...while

30) An infinite loop occurs when the loop's stopping condition never becomes false.

Eg) let i = 0;
while (i < 5) {

console.log(i);
}

The value of i is never updated, so i < 5 remains true forever. Fix:

let i = 0;
while (i < 5) {
console.log(i);
i++;
}

Coding:

31) let num = -5;

if (num > 0) {
console.log("positive");

} else if (num < 0) {
console.log("Negative");

```
3 else {  
    console.log("zero");  
}
```

3

```
32) let num = 5;  
switch (Math.sign(num)) {
```

```
    case 1:
```

```
        console.log("positive");  
        break;
```

```
    case -1:
```

```
        console.log("Negative");  
        break;
```

```
    case 0:
```

```
        console.log("zero");  
        break;
```

```
    default:
```

```
        console.log("Invalid number")  
}
```

```
33) let age = 20;  
let citizen = true;
```

```
if (age >= 18) {
```

```
    if (citizen === true) {
```

```
        console.log("Eligible to vote");  
    }
```

```
    } else {
```

```
        console.log("Not Eligible. Must be 18 or older");  
    }
```

3

```
34) for (let i = 1, i <= 20, i++) {
```

```
    if (i % 3 === 0) {
```

Continue;

```
}  
console.log(i);
```

```
}
```

O/P

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

35) let numbers = [10, 25, 40, 55, 70, 80];

```
for(let number of numbers) {
```

```
  if (number > 50) {
```

```
    console.log("First number greater than 50:", number);
```

```
    break;
```

```
  }
```

```
}
```

O/P: First number greater than

50: 55

36) let day = "Saturday";

```
switch(day) {
```

```
  case "Monday":
```

```
  case "Tuesday":
```

```
  case "Wednesday":
```

```
  case "Thursday":
```

```
  case "Friday":
```

```
console.log("Weekday");
```

```
break;
```

```
case "Saturday"
```

```
case "Sunday"
```

```
console.log("Week end");
```

```
break;
```

```
default;
```

```
console.log("Invalid day");
```

```
}
```

Here the multiple cases intentionally fall through & share the same code.

37) let score = 85;

```
if (score >= 90) {
```

```
    console.log("A");
```

```
} else if (score >= 80) {
```

```
    console.log("B");
```

```
} else if (score >= 70) {
```

```
    console.log("C");
```

```
} else if (score >= 60) {
```

```
    console.log("D");
```

```
} else {
```

```
    console.log("F");
```

```
}
```

For score = 85 the o/p is B

38) function validatePassword (password) {

if (password.length >= 8) {

if (/^[A-Z]/.test(password)) {

if (/^[0-9]/.test(password)) {

return "Valid password";

} else {

return "Password must contain a number";

}

} else {

return "Password must contain an uppercase letter";

}

} else {

return "Password must be at least 8 characters long";

}

}

console.log(validatePassword("Hello123"));

O/P:

Valid password

39) let age = 25;

switch (true) {

case age < 13:

console.log("child");

break;

case age < 18;

console.log("Teenager");

break;

case age < 60

```
console.log("Adult");  
break;
```

~~case~~
default

```
console.log("Senior");
```

For age = 25 the o/p is Adult.

40) let numbers = [10, -5, 20, -3, 15];

let sum = 0;

```
for (let number of numbers) {
```

```
  if (number < 0) {
```

```
    continue;
```

```
  }
```

```
    sum += number;
```

```
  }
```

```
  console.log(sum);
```

o/p: 45

Negative numbers are skipped using continue.

Real world scenarios;

1) Switch is cleaner because the traffic light has a fixed set of known states "red", "yellow", and "green". Each state can have its own case, making the code easy to read and maintain.

2) Switch is generally suitable when the zones are fixed discrete values such as zone 1, zone 2, zone 3 etc. As

the number of fixed zones grows, switch keeps each zone's logic clearly separated. If zones are based on ranges or more complex rules, if/else if may be more appropriate.

43) A single if statement using the logical && operator is appropriate:
if(emailValid && passwordStrong && termsAccepted) {
 // Valid

Deep nesting is risky because it makes the code harder to read, maintain, and debug. Combining independent Boolean conditions or using early return can keep the logic flatter.

44) Use continue.

Continue skip processing the current paused frame and moves to the next iteration. break would terminate the entire loop.

45) Use break.

break immediately terminates the loop once the required item is found, avoiding unnecessary processing.

46) Switch can be easier to read because the rule is one value with several fixed possible values.

For eg) admin
editor
viewer

Each role can have its own case, making the role-based

logic visually clear and easy to maintain.

47) Deep nesting makes the control flow difficult to understand and increases the chance of bugs during maintenance. A common technique is using early returns/guard clauses instead of

```
if (user) {  
    if (user.active) {  
        // code  
    }  
}
```

```
if (user) return;  
if (!user.active) return;  
// code
```

This reduces nesting and makes the main logic easier to follow.

48) Fallthrough allows multiple plan cases to share the same logic. For eg) if pro and enterprise receive the same feature.

```
Switch (plan) {
```

```
    case "pro":
```

```
        case "enterprise"
```

```
            console.log("premium features");  
            break;
```

```
        case "free":
```

```
            console.log("Basic features");  
            break;  
}
```

Both "pro" and "enterprise" intentionally fall through to the same block.

49) A loop is used for the retry attempts, and break is used to stop the loop immediately after a successful attempt. Conceptually,

Attempt 1 → fail

Attempt 2 → fail

Attempt 3 → success → break

Continue could also be used when explicitly skipping to the next retry after a failed attempt, but break is the key keyword for stopping after success.

50) Switch (true) allows condition expressions to be placed in individual case clauses, making several Boolean conditions structurally similar to a switch.

However the tradeoff is readability. Although it is valid Javascript. Ordinary if/else if is usually more natural and immediately understandable for conditions based logic. Therefore, switch (true) can be useful in some code bases, but it should not be used merely for the sake of using switch.